Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Парфенюк Д. П.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Задание 1:
17) База данных Ресурсы Internet:

Решение:

```
"""
Модуль для лабораторной работы №5.
Вариант 17: Разработка API и базы данных "Ресурсы Internet".
"""

# pylint: disable=too-few-public-methods, invalid-name

from typing import List, Optional
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey,
Float, Table
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session, relationship
from pydantic import BaseModel

# --- НАСТРОЙКА БД ---
DATABASE_URL = "sqlite:///./internet_resources.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})
SESSION_LOCAL = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# --- МОДЕЛИ SQLALCHEMY ---
resource_tags = Table(
    "resource_tags",
    Base.metadata,
    Column("resource_id", ForeignKey("resources.id"), primary_key=True),
    Column("tag_id", ForeignKey("tags.id"), primary_key=True),
)


class Category(Base):
    """Модель таблицы категорий."""

    __tablename__ = "categories"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, unique=True)
    resources = relationship("Resource", back_populates="category")


class Resource(Base):
    """Модель таблицы ресурсов (сайтов)."""

    __tablename__ = "resources"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    url = Column(String)
    description = Column(String)
    category_id = Column(Integer, ForeignKey("categories.id"))

    category = relationship("Category", back_populates="resources")
    reviews = relationship("Review", back_populates="resource")
    tags = relationship("Tag", secondary=resource_tags,
back_populates="resources")
```

```python
class User(Base):
    """Модель таблицы пользователей."""

    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True)
    email = Column(String)
    reviews = relationship("Review", back_populates="user")


class Review(Base):
    """Модель таблицы отзывов."""

    __tablename__ = "reviews"
    id = Column(Integer, primary_key=True, index=True)
    rating = Column(Float)
    comment = Column(String)
    user_id = Column(Integer, ForeignKey("users.id"))
    resource_id = Column(Integer, ForeignKey("resources.id"))

    user = relationship("User", back_populates="reviews")
    resource = relationship("Resource", back_populates="reviews")


class Tag(Base):
    """Модель таблицы тегов."""

    __tablename__ = "tags"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, unique=True)
    resources = relationship("Resource", secondary=resource_tags,
back_populates="tags")


Base.metadata.create_all(bind=engine)


# --- СХЕМЫ PYDANTIC ---
class CategoryCreate(BaseModel):
    """Схема для создания категории."""

    name: str


class CategoryOut(BaseModel):
    """Схема для вывода данных категории."""

    id: int
    name: str

    class Config:
        """Конфигурация Pydantic."""

        from_attributes = True


class ResourceBase(BaseModel):
    """Базовая схема ресурса."""

    title: str
    url: str
    description: Optional[str] = None
    category_id: int
```

```python
class ResourceCreate(ResourceBase):
    """Схема для создания ресурса."""


class ResourceOut(ResourceBase):
    """Схема для вывода данных ресурса."""

    id: int

    class Config:
        """Конфигурация Pydantic."""

        from_attributes = True


# --- FASTAPI APP ---
app = FastAPI(title="Internet Resources API")


def get_db():
    """Зависимость для получения сессии базы данных."""
    database = SESSION_LOCAL()
    try:
        yield database
    finally:
        database.close()


@app.post("/categories/", response_model=CategoryOut)
def create_category(category: CategoryCreate, db: Session = Depends(get_db)):
    """Добавление новой категории."""
    db_cat = Category(name=category.name)
    db.add(db_cat)
    db.commit()
    db.refresh(db_cat)
    return db_cat


@app.get("/categories/", response_model=List[CategoryOut])
def read_categories(db: Session = Depends(get_db)):
    """Получение всех категорий."""
    return db.query(Category).all()


@app.post("/resources/", response_model=ResourceOut)
def create_resource(resource: ResourceCreate, db: Session = Depends(get_db)):
    """Добавление нового ресурса."""
    cat = db.query(Category).filter(Category.id ==
resource.category_id).first()
    if not cat:
        raise HTTPException(status_code=400, detail="Category not found.")

    db_resource = Resource(**resource.model_dump())
    db.add(db_resource)
    db.commit()
    db.refresh(db_resource)
    return db_resource


@app.get("/resources/", response_model=List[ResourceOut])
def read_resources(db: Session = Depends(get_db)):
    """Получение всех ресурсов."""
    return db.query(Resource).all()
```

```python
@app.delete("/resources/{resource_id}")
def delete_resource(resource_id: int, db: Session = Depends(get_db)):
    """Удаление ресурса по ID."""
    db_resource = db.query(Resource).filter(Resource.id ==
resource_id).first()
    if not db_resource:
        raise HTTPException(status_code=404, detail="Resource not found")
    db.delete(db_resource)
    db.commit()
    return {"message": "Deleted"}


if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host="127.0.0.1", port=8000)
```

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/resources/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "title": "Google",
  "url": "https://google.com",
  "description": "поисковая система",
  "category_id": 1
}'
```

Request URL

```
http://127.0.0.1:8000/resources/
```

Server response

Code	Details
200	Response body

```
{
  "title": "Google",
  "url": "https://google.com",
  "description": "поисковая система",
  "category_id": 1,
  "id": 1
}
```

Download

Response headers

```
content-length: 118
content-type: application/json
date: Mon,30 Mar 2026 16:44:55 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "title": "string",
  "url": "string",
  "description": "string",
  "category_id": 0,
  "id": 0
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/categories/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "поисковики"
}'
```

Request URL

```
http://127.0.0.1:8000/categories/
```

Server response

Code	Details
200	Response body

```
{
  "id": 1,
  "name": "поисковики"
}
```

Response headers

```
content-length: 38
content-type: application/json
date: Mon,30 Mar 2026 16:42:51 GMT
server: uvicorn
```

Responses

Code	Description	Link
200	Successful Response	No li

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "name": "string"
}
```

Имя	Тип	Схема
Таблицы (6)		
categories		CREATE TABLE categories (id INTEGER NOT NULL, name VARCHAR, PRIMARY KEY (id), UNIQUE (name))
resource_tags		CREATE TABLE resource_tags (resource_id INTEGER NOT NULL, tag_id INTEGER NOT NULL, PRIMARY KEY (resource_id, tag_id), FOREIGN KEY(resource_id) REF
resources		CREATE TABLE resources (id INTEGER NOT NULL, title VARCHAR, url VARCHAR, description VARCHAR, category_id INTEGER, PRIMARY KEY (id), FOREIGN KEY(c
reviews		CREATE TABLE reviews (id INTEGER NOT NULL, rating FLOAT, comment VARCHAR, user_id INTEGER, resource_id INTEGER, PRIMARY KEY (id), FOREIGN KEY(us
tags		CREATE TABLE tags (id INTEGER NOT NULL, name VARCHAR, PRIMARY KEY (id), UNIQUE (name))
users		CREATE TABLE users (id INTEGER NOT NULL, username VARCHAR, email VARCHAR, PRIMARY KEY (id), UNIQUE (username))
Индексы (6)		
ix_categories_id		CREATE INDEX ix_categories_id ON categories (id)
ix_resources_id		CREATE INDEX ix_resources_id ON resources (id)
ix_resources_title		CREATE INDEX ix_resources_title ON resources (title)
ix_reviews_id		CREATE INDEX ix_reviews_id ON reviews (id)
ix_tags_id		CREATE INDEX ix_tags_id ON tags (id)
ix_users_id		CREATE INDEX ix_users_id ON users (id)
Представления (0)		
Триггеры (0)		

Вывод: приобрел практические навыки разработки API и баз данных